

REACT · TYPESCRIPT · STYLED-COMPONENTS

Como pensar e construir o ProductDetail

Um percurso visual da URL aos pixels, usando o Crimson Ronin como exemplo concreto.



Mapa visual da referência: 1) miniaturas, 2) foto principal, 3) conteúdo, 4) ações.

Material criado a partir do código real do frontend Save Point3D. O objetivo não é entregar uma receita cega, e sim ensinar a decompor a tela, escolher a estrutura, entender a sintaxe e depurar o resultado.

Como usar este guia

A referência visual parece uma única imagem, mas o navegador não a enxerga assim. Ele recebe uma árvore de elementos, calcula a dimensão de cada caixa, escolhe quais pixels da fotografia ficam visíveis e só depois pinta textos, bordas e botões. Por isso, a maneira mais segura de recriar o resultado é percorrer as camadas na ordem em que os dados e o navegador trabalham: primeiro descobrir qual produto a URL escolheu; depois construir uma árvore semântica; em seguida distribuir as regiões com Grid; por fim adicionar estados e detalhes visuais. Sempre que você se sentir perdido em uma margem ou tamanho, volte a esse encadeamento.

O mapa de raciocínio: cada camada responde a uma pergunta

Não comece pela margem. Comece pelo caminho do dado e pela hierarquia visual.

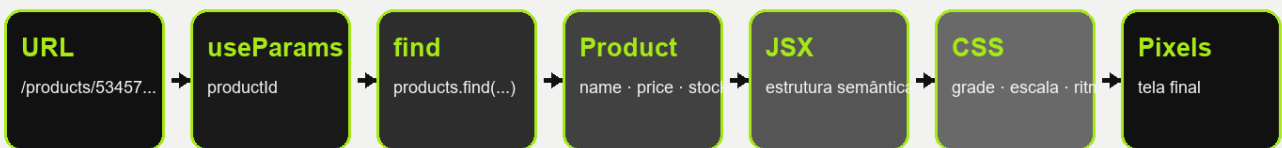


Figura 1. O caminho completo do valor productId até a tela pintada.

Roteiro

1. Ler a referência como regiões e responsabilidades
2. Diagnosticar o componente atual sem apagar o que ele já ensina
3. Entender rota, parâmetro e seleção do produto
4. Definir o contrato de dados que a interface realmente exige
5. Desenhar a árvore JSX antes de estilizar
6. Construir a grade principal e compreender fr, minmax e vírgulas
7. Implementar a galeria e o recorte da fotografia
8. Montar o painel comercial e formatar preço
9. Fazer quantidade e carrinho compartilharem o mesmo significado
10. Adaptar desktop, tablet e celular
11. Acessibilidade, estados de carregamento e validação
12. Sequência prática de implementação e checklist final

1. Leia a referência antes de escrever CSS

Comece fazendo uma pergunta simples: quais decisões visuais esta tela precisa comunicar? A grande foto à esquerda vende a peça pela presença; a coluna de miniaturas comunica que existem outras vistas; o painel direito organiza nome, categoria, escala, disponibilidade, avaliação, explicação, preço e ações. Essa leitura transforma uma imagem aparentemente complexa em dois grandes domínios: galeria e informação comercial. Dentro deles existem sub-regiões menores. A primeira decisão de implementação, portanto, não é escolher `padding: 24px`; é dar nomes a essas responsabilidades.

Na referência, o lado esquerdo ocupa aproximadamente 46% da largura e o painel direito aproximadamente 54%. A miniatura é uma coluna fixa dentro da galeria. Essa relação sugere uma grade externa com duas colunas flexíveis e uma grade interna com uma coluna estreita para miniaturas. A diferença é importante: tentar resolver tudo em uma única grade cria linhas e colunas que não representam a árvore de conteúdo; separar as grades permite que a galeria mude internamente sem afetar preço e ações.



Figura 2. A referência reconstruída como quatro regiões numeradas, usando o asset real <public/assets/img/1.png>.

Mapa mental

Se você consegue apontar uma região e explicar sua responsabilidade com uma frase, ela provavelmente merece um elemento próprio no JSX. Se não consegue explicar a responsabilidade, talvez seja apenas decoração CSS.

2. O que o código atual está tentando fazer - e por que ainda não encaixa

Seu componente já encontrou o produto pela URL, já renderiza a imagem e já chama `onAddToCart`. Isso significa que a conexão principal existe. O obstáculo não é falta de capacidade; é que estrutura e estilo ainda estão misturados com conceitos de outro componente. O `ProductDetail` importa `styles` do `ProductCard`, então nomes como `imageWrap`, `tag` e `addButton` pertencem a uma superfície visual que tem proporções e comportamento diferentes. Reutilizar lógica é desejável; reutilizar classes apenas porque parecem convenientes costuma criar acoplamento invisível.

Quatro pontos que impedem o layout atual de chegar à referência

1 `className={ProductStyle}`

ProductStyle é um componente React, não o nome de uma classe CSS.

2 `1fr, 1fr`

A vírgula torna o valor inválido. CSS Grid separa colunas por espaço.

3 `height: 20%`

A porcentagem depende da altura do pai. Sem altura definida, o resultado é instável.

4 `styles do ProductCard`

A tela detalhada fica acoplada a outro componente e perde vocabulário próprio.

Figura 3. Diagnóstico visual dos quatro pontos que devem ser corrigidos antes do refinamento.

O trecho `` mistura dois mecanismos. `ProductStyle` foi criado por `styled.div`; portanto, ele é um componente React que deve aparecer como tag: `...`. Já `className` exige uma string, como `"productPage"`. O objeto `stylesProduct` resolve justamente essa tradução ao guardar strings. Em linguagem natural: a tag styled-component cria o escopo; as strings do objeto identificam elementos internos desse escopo.

```
// Incorreto: ProductStyle não é uma string de classe.
<section className={ProductStyle}>...</section>

// Correto: ProductStyle é a própria tag; page é uma string.
<ProductStyle>
  <main className={stylesProduct.page}>...</main>
</ProductStyle>
```

A segunda falha é sintática: `grid-template-columns: 1fr, 1fr` contém uma vírgula. Em CSS Grid, as trilhas são separadas por espaço: `1fr 1fr`. `fr` significa uma fração do espaço livre; duas vezes `1fr` dividem igualmente. Como o valor com vírgula não pertence à gramática da propriedade, o navegador o ignora. Quando uma regra parece não surtir efeito, inspecione se ela foi riscada no DevTools; isso costuma indicar valor inválido ou sobrescrito.

3. Da URL ao produto: seguindo um valor concreto

Considere a URL concreta `/products/53457ac5-f790-406a-9e56-ed30255086bf`. Na declaração da rota, `:productId` possui dois sinais importantes. Os dois pontos, `:`, dizem ao React Router que aquele segmento não é texto fixo; ele é variável. `productId` é o nome escolhido para recuperar esse valor. Por isso o nome usado em `useParams<{ productId: string }>()` deve coincidir exatamente com o nome escrito depois dos dois pontos. Se a rota declarasse `:id` e o hook procurasse `productId`, o resultado seria `undefined`.

```
// routes/index.tsx
<Route
  path="/products/:productId"
  element=<ProductDetail products={products} onAddToCart={onAddToCart} />
/>

// hooks/useUrlParser.tsx
const { productId } = useParams<{ productId: string }>();
const selectedProduct = products.find(
  (product) => product.id === productId,
);
```

`useParams` devolve um objeto porque uma rota pode ter vários parâmetros. A desestruturação `{ productId }` retira apenas a propriedade de interesse. O trecho `<{ productId: string }>` é um argumento genérico de TypeScript: ele descreve o formato esperado, mas não cria o valor em execução. Em seguida, `find` percorre o array `products`; para cada objeto, a função seta compara `product.id` com o texto recebido da URL. O operador `===` exige igualdade de valor e tipo. Quando encontra a primeira correspondência, `find` devolve o objeto; se não encontra, devolve `undefined`.

O mapa de raciocínio: cada camada responde a uma pergunta

Não comece pela margem. Comece pelo caminho do dado e pela hierarquia visual.



Figura 4. `productId` nasce na URL, é capturado pelo hook, comparado aos IDs e termina nos elementos visuais.

Estado de carregamento

No App, `products` começa como `[]` e só é preenchido depois do `fetch`. Portanto, `undefined` pode significar “a API ainda não respondeu” ou “o produto realmente não existe”. Um `ProductDetail` robusto recebe `loading` ou usa um estado de consulta que distingue `loading`, `success` e `error`; não deve mostrar “Produto não encontrado” durante os primeiros milissegundos.

4. O contrato de dados deve nascer da interface

A interface atual `Product` possui nome, categoria, escala, material, preço, estoque e uma única `imageUrl`. A referência, porém, mostra descrição, avaliação, preço anterior, parcelamento e várias imagens. CSS não consegue inventar esses valores. Antes de desenhar estrelas ou miniaturas, você precisa decidir de onde cada dado virá. Em produção, o caminho correto é ampliar o contrato do backend e normalizá-lo no frontend. Durante uma fase exclusivamente visual, é aceitável usar fallbacks explícitos, desde que eles sejam identificados como provisórios e não pareçam dados reais.

```
export interface Product {
  id: string;
  name: string;
  price: number;
  stock: number | null;
  imageUrl: string;
  images?: string[];
  description?: string | null;
  rating?: number | null;
  originalPrice?: number | null;
  installments?: number | null;
  // ...campos existentes
}

const gallery = product.images?.length
  ? product.images
  : [product.imageUrl];
```

O símbolo `?` depois do nome de uma propriedade significa que ela é opcional: objetos antigos podem não tê-la. Já `string | null` significa que a propriedade existe, mas pode declarar ausência de conteúdo. Em `product.images?.length`, `?.` é o encadeamento opcional: se `images` for `undefined`, a expressão para sem lançar erro. O operador ternário possui a forma `condição ? valorSeVerdadeiro : valorSeFalso`. Aqui ele escolhe a galeria do backend quando há itens e recua para a imagem principal quando ainda não há galeria.

Origem e destino

`items.price` chega como string da API; `fetchProducts` o converte com `Number`; o `ProductDetail` recebe um number; `Intl.NumberFormat` transforma esse número em texto localizado; o texto termina no elemento visual do preço. Cada transformação tem um motivo e uma camada responsável.

5. Escreva a árvore JSX antes das regras visuais

JSX não é uma imagem do resultado: é uma descrição hierárquica. A tag `main` identifica o conteúdo principal da rota; `section` agrupa a galeria; `nav` informa que as miniaturas são controles de navegação entre imagens; `figure` contém a mídia principal; `article` reúne informação autossuficiente sobre o produto; `button` representa uma ação real e recebe foco de teclado. Essa semântica ajuda leitores de tela, testes e manutenção, mas também melhora seu pensamento: cada tag passa a ter uma responsabilidade verificável.

A árvore visual antes do CSS



Figura 5. Árvore DOM sugerida. A indentação mostra quem contém quem.

```
<ProductStyle>
  <main className={styles.page}>
    <section className={styles.gallery} aria-label="Galeria do produto">
      <nav className={styles.thumbnails} aria-label="Imagens do produto">
        {gallery.map((image, index) => (/ * botão + miniatura */))}
      </nav>
      <figure className={styles.hero}>
        <img src={activeImage} alt={product.alt || product.name} />
      </figure>
    </section>

    <article className={styles.info}>
      <header>{/ * meta, estoque, título e avaliação */}</header>
      <section className={styles.priceBlock}>{/ * preço */}</section>
      <section className={styles.actions}>{/ * quantidade e CTAs */}</section>
    </article>
  </main>
</ProductStyle>
```

As chaves `{}` dentro do JSX mudam do modo marcação para o modo JavaScript. Por isso `{product.name}` lê uma variável, enquanto `product.name` escrito sem chaves seria texto literal. Comentários em JSX usam `{/\* ... \*/}` porque também precisam entrar no modo JavaScript. O fechamento `>` indica um elemento sem filhos; o fechamento `` encerra um elemento que contém outros nós.

6. Faça o Grid resolver a macroestrutura

Use Grid para relações bidimensionais estáveis: galeria ao lado de informação, miniaturas ao lado da foto. Use Flexbox quando a relação principal for uma linha ou coluna de conteúdo: meta com estoque, seletor com botões, texto do preço. Essa distinção evita usar uma ferramenta por hábito. A grade externa precisa permitir que o lado da fotografia encolha sem criar overflow e, ao mesmo tempo, proteger uma largura mínima confortável para o painel de leitura.

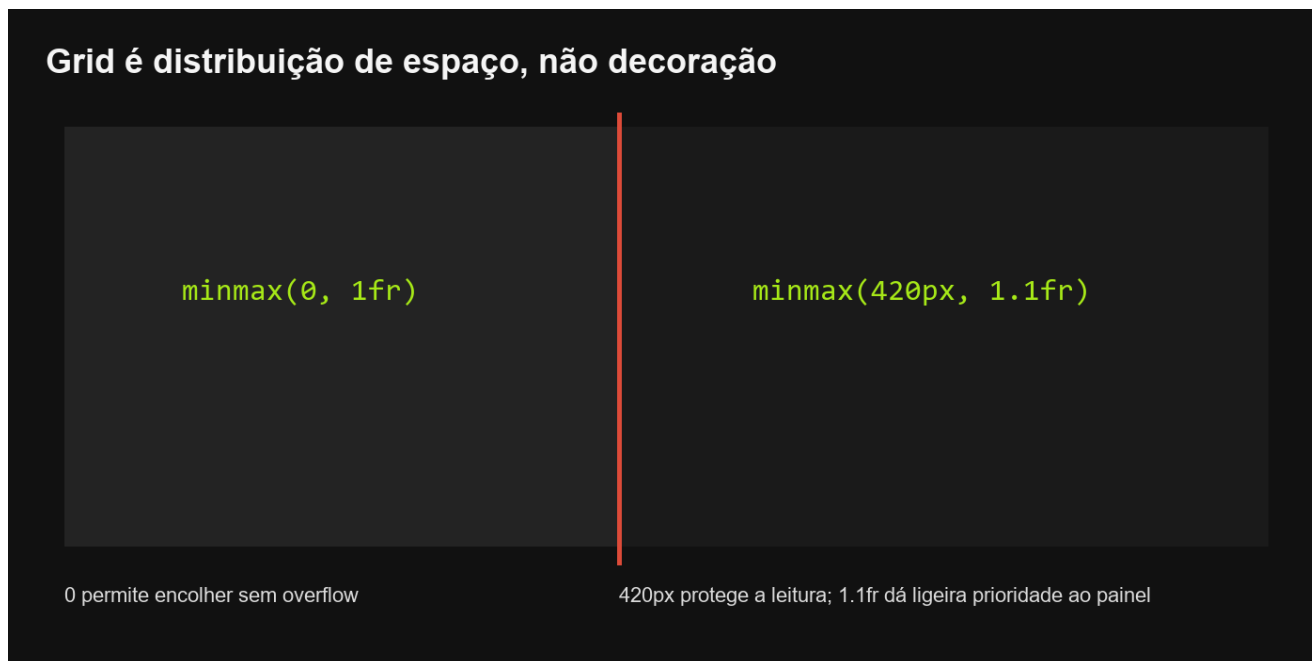


Figura 6. O significado de cada trilha na grade principal.

```
.page {  
  display: grid;  
  grid-template-columns: minmax(0, 1fr) minmax(420px, 1.1fr);  
  min-height: min(820px, calc(100vh - 80px));  
  background: var(--color-ink-soft);  
}  
  
.gallery {  
  display: grid;  
  grid-template-columns: 132px minmax(0, 1fr);  
  min-width: 0;  
}
```

‘minmax(mínimo, máximo)’ cria uma trilha com limites. ‘minmax(0, 1fr)’ diz: esta coluna pode encolher até zero na matemática do Grid e recebe uma fração do espaço livre. O zero evita que o tamanho mínimo implícito do conteúdo empurre a página para fora da viewport. Na segunda coluna, ‘420px’ protege a legibilidade do painel e ‘1.1fr’ lhe dá uma fração ligeiramente maior. Não existe vírgula entre colunas; o espaço separa as duas trilhas. Já dentro de ‘minmax’, a vírgula separa os dois argumentos da função CSS.

Regra de depuração

Primeiro pinte cada região com uma cor temporária. Se as caixas não ocupam o lugar certo, não ajuste fonte, sombra ou borda. Resolva macroestrutura, depois espaçamento, depois tipografia e somente no fim os detalhes decorativos.

7. Construa a galeria como estado + lista + enquadramento

A galeria possui dois tipos de estado: as imagens disponíveis, que vêm do produto, e a imagem ativa, que muda com a interação. `useState(gallery[0])` cria uma memória local para a URL escolhida. A função `setActiveImage` é a única maneira de pedir ao React uma nova renderização com outro valor. Quando a pessoa clica numa miniatura, o evento `onClick` chama uma função seta que guarda aquela URL; na renderização seguinte, o `src` da imagem principal usa o novo estado.

Galeria em três decisões independentes

A. Lista

map() cria um botão por imagem



button
> img

B. Seleção

activeImage guarda a URL escolhida

active



C. Enquadramento

object-fit: cover ocupa o quadro



width: 100%; height: 100%

Figura 7. Três decisões separadas: criar controles, guardar seleção e recortar a foto.

```
const [activeImage, setActiveImage] = useState(gallery[0]);

{gallery.map((image, index) => (
  <button
    key={` ${image}-${index}`}
    type="button"
    className={image === activeImage ? styles.thumbnailActive : styles.thumbnail}
    onClick={() => setActiveImage(image)}
    aria-label={`Mostrar imagem ${index + 1} de ${product.name}`}
    aria-pressed={image === activeImage}
  >
    <img src={image} alt="" />
  </button>
)}
)}
```

`map` transforma cada item do array em um elemento React. A propriedade `key` dá identidade estável ao item para que o React compare listas; ela não é exibida no HTML. O template literal usa crases e `\${...}` para inserir valores. `aria-pressed` comunica o estado selecionado às tecnologias assistivas. A miniatura recebe `alt=""` porque o botão já possui um nome acessível completo; repetir a descrição da imagem faria o leitor de tela anunciar informação duplicada.

```
.hero { min-height: 680px; overflow: hidden; background: #070707; }
.hero img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  object-position: center;
  display: block;
}
```

`object-fit: cover` preserva a proporção da fotografia e corta a sobra necessária para preencher o quadro. `contain` preservaria a imagem inteira, mas poderia criar faixas vazias. `display: block` remove a pequena lacuna de linha-base que imagens inline deixam abaixo de si. A escolha entre cover e contain não é puramente técnica: cover reproduz a referência mais imersiva; contain é mais apropriado quando nenhum detalhe do produto pode ser cortado.

8. Monte o painel comercial na ordem da decisão

O painel não deve ser uma coleção aleatória de textos. Ele guia uma sequência cognitiva: identificar, verificar disponibilidade, construir confiança, entender valor e agir. Tamanho, peso e contraste expressam essa ordem. O título é o maior texto; a categoria é menor e cinza; o estoque usa a cor de destaque; a descrição usa largura limitada para não criar linhas longas; o preço volta a ganhar escala; os botões ocupam a largura disponível para sinalizar ação.

Hierarquia comercial: responder na ordem em que a pessoa decide

1

O que é?

CRIMSON RONIN

2

Está disponível?

5 UNI.

3

Por que confiar?

Avaliação ☐☐☐☐

4

Quanto custa?

R\$ 899,00

5

O que faço agora?

ADICIONAR / COMPRAR

Figura 8. Hierarquia de perguntas que o painel deve responder.

```
const money = new Intl.NumberFormat("pt-BR", {
  style: "currency",
  currency: "BRL",
});

<span className={styles.meta}>
  {product.categoryLabel} · Escala {product.scale}
</span>
<h1 className={styles.title}>{product.name}</h1>
<p className={styles.description}>{product.description}</p>
<strong className={styles.price}>{money.format(product.price)}</strong>
```

`new Intl.NumberFormat` cria um formatador consciente de localidade. `pt-BR` seleciona convenções brasileiras; `style: "currency"` pede formato monetário; `currency: "BRL"` define real brasileiro. O objeto entre chaves reúne opções nomeadas. Diferentemente de `toFixed(2).replace('.', ',')`, essa API também resolve símbolo, separador de milhar, casas decimais e espaços segundo a localidade.

```
.info { padding: clamp(36px, 5vw, 76px); color: #f2f2f0; }
.title {
  margin: 22px 0 14px;
  font-size: clamp(42px, 5vw, 76px);
  line-height: .94;
```

```

text-transform: uppercase;
}
.description { max-width: 64ch; color: #dddddd; line-height: 1.45; }
.price { display: block; font-size: clamp(46px, 5vw, 72px); }

```

`clamp(mínimo, preferido, máximo)` cria tipografia fluida sem permitir extremos. `64ch` limita a linha a aproximadamente 64 caracteres com base na largura do caractere zero da fonte, oferecendo uma medida mais ligada à leitura do que pixels fixos. `line-height: .94` aproxima linhas de um título grande; na descrição, `1.45` abre espaço para leitura contínua.

9. Quantidade só está pronta quando chega ao carrinho

Um seletor visual que mostra `2` mas chama `onAddToCart(id)` apenas uma vez cria duas verdades diferentes: a tela afirma uma quantidade, o estado global recebe outra. O contrato da função precisa carregar o mesmo significado que a interface mostra. A opção recomendada é evoluir `onAddToCart` para receber `quantity`; o componente detalhado decide a quantidade desejada e o `App` continua responsável por validar estoque e atualizar o carrinho.

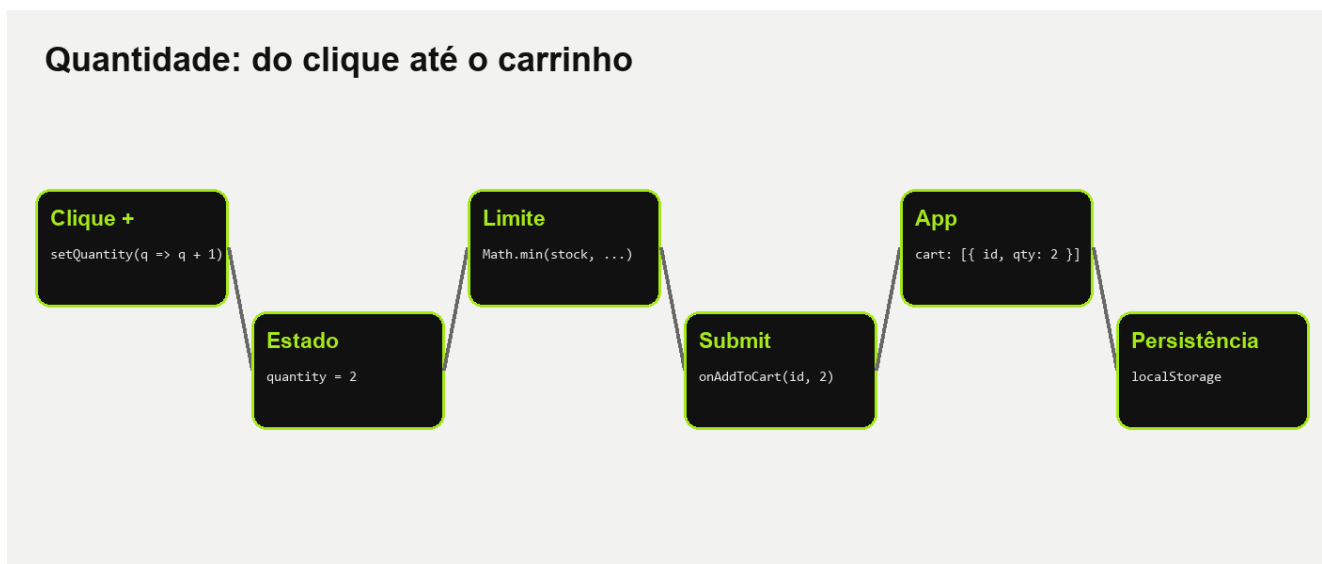


Figura 9. O valor quantity precisa atravessar todas as camadas até CartLine.qty.

```

interface ItemProps {
  products: Product[];
  onAddToCart: (id: string, quantity: number) => void;
}

const [quantity, setQuantity] = useState(1);
const maxQuantity = product.stock ?? Number.POSITIVE_INFINITY;

function increment() {
  setQuantity((current) => Math.min(maxQuantity, current + 1));
}

function decrement() {
  setQuantity((current) => Math.max(1, current - 1));
}

function addSelectedQuantity() {
  onAddToCart(product.id, quantity);
}

```

A forma `setQuantity((current) => ...)` é uma atualização funcional. React entrega o valor mais recente a `current`, evitando depender de uma captura antiga quando vários cliques são processados próximos. `Math.min` impede ultrapassar o estoque; `Math.max` impede cair abaixo de um. O operador `??` usa o valor da direita apenas quando o da esquerda é `null` ou `undefined`; aqui, estoque nulo representa disponibilidade sem limite numérico.

```
function addToCart(id: string, amount = 1) {
  const product = products.find((item) => item.id === id);
  if (!product || product.stock === 0) return;

  setCart((current) => {
    const existing = current.find((line) => line.id === id);
    const desired = (existing?.qty ?? 0) + amount;
    const nextQty = product.stock === null
      ? desired
      : Math.min(desired, product.stock);

    return existing
      ? current.map((line) => line.id === id ? { ...line, qty: nextQty } : line)
      : [...current, { id, qty: nextQty }];
  });
}
```

`...line` é o operador spread: copia as propriedades do objeto existente antes de substituir `qty`. `...current` copia os elementos do array antes de anexar uma nova linha. Esse padrão mantém imutabilidade: em vez de alterar o objeto anterior, cria um novo valor, permitindo que React detecte a mudança. A regra de estoque continua no `App`, porque todo ponto de entrada no carrinho deve obedecer à mesma restrição.

10. Responsividade é reorganizar prioridades

Reduzir tudo proporcionalmente não produz uma boa tela móvel. Em largura estreita, fotografia e informação não cabem lado a lado; a decisão correta é mudar a ordem espacial sem mudar a ordem semântica. A galeria vem primeiro para manter contexto, o painel vem depois, as miniaturas passam de coluna vertical para faixa horizontal e os botões ocupam largura total. O título usa `clamp`; o padding diminui; a altura da foto passa a ser controlada por `aspect-ratio`.

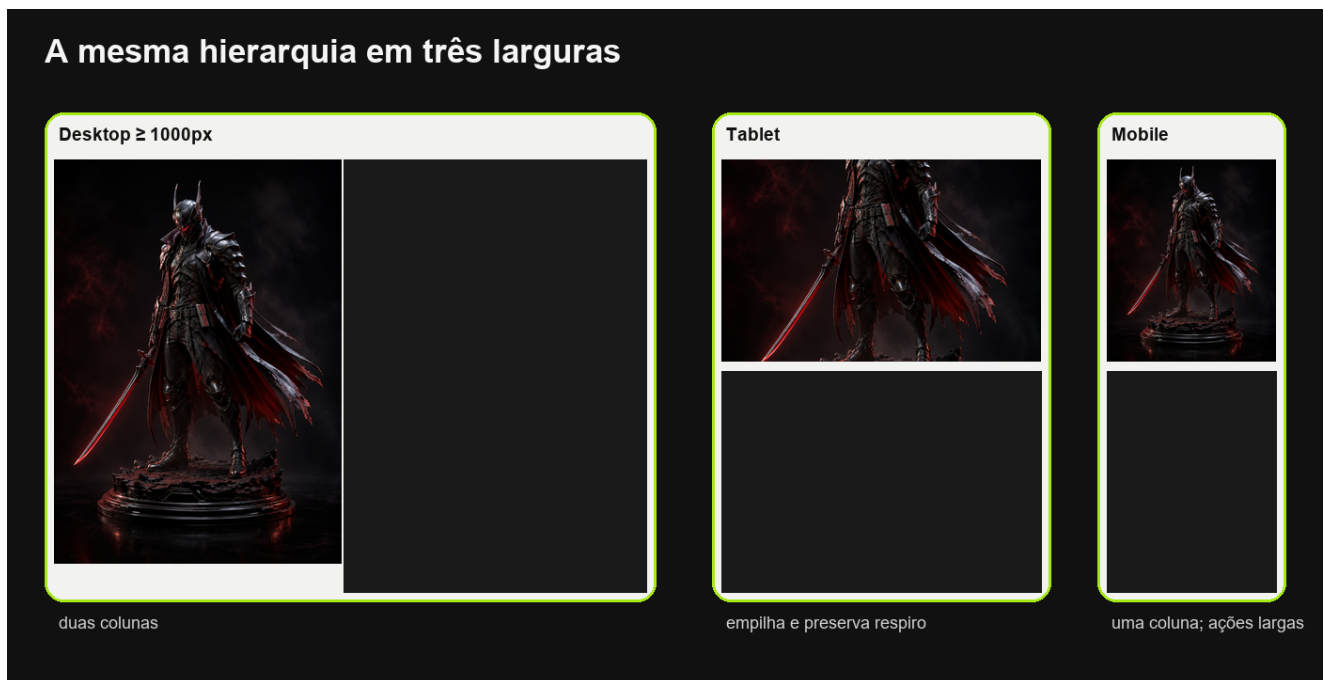


Figura 10. Três composições para o mesmo conteúdo, sem duplicar JSX.

```
@media (max-width: 1000px) {
  .page { grid-template-columns: 1fr; }
  .gallery { min-height: auto; }
  .hero { aspect-ratio: 4 / 5; min-height: 0; }
}

@media (max-width: 700px) {
  .gallery { grid-template-columns: 1fr; }
  .thumbnails {
    grid-row: 2;
    display: flex;
    overflow-x: auto;
    padding: 10px;
  }
}
```

```

    }
    .thumbnail { flex: 0 0 82px; aspect-ratio: 4 / 5; }
    .info { padding: 32px 22px 48px; }
    .actionRow { grid-template-columns: 1fr; }
  }

```

`@media (max-width: 700px)` ativa as regras apenas quando a viewport tem no máximo 700 pixels. `grid-row: 2` move as miniaturas para a segunda linha da grade. `overflow-x: auto` cria rolagem horizontal somente quando necessária. `flex: 0 0 82px` significa: não crescer, não encolher e usar base de 82 pixels. O ponto de quebra deve ser escolhido quando o conteúdo deixa de caber, não apenas por um modelo específico de aparelho; neste projeto, 700px coincide com o token de mobile já existente.

11. Estados, acessibilidade e depuração

Uma página detalhada não possui apenas o estado “produto encontrado”. Ela pode estar carregando, falhar na API, receber um ID inexistente, ter estoque zero ou imagem quebrada. Esses estados precisam ser modelados antes do acabamento. Enquanto carrega, mantenha uma estrutura de altura semelhante para evitar salto de layout. Em erro, ofereça uma mensagem e caminho de volta. Em estoque zero, desabilite ações e explique o motivo. Para imagens, use `alt` significativo na foto principal e `alt=""` nas miniaturas cujo botão já possui nome acessível.

```

if (loading) {
  return <ProductDetailSkeleton aria-label="Carregando produto" />;
}

if (error) {
  return <StatusPage title="Não foi possível carregar o produto" />;
}

if (!product) {
  return <StatusPage title="Produto não encontrado" />;
}

```

Botões de `–` e `+` precisam de `aria-label`, porque o caractere isolado pode não explicar a ação. A miniatura ativa recebe `aria-pressed`. O foco visível não deve ser removido; use `:focus-visible` para desenhar contorno verde quando a navegação ocorre por teclado. A preferência global `prefers-reduced-motion` já desativa transições no projeto, então qualquer animação nova deve continuar sob esse escopo, em vez de ignorá-lo.

```

.button:focus-visible,
.thumbnail:focus-visible {
  outline: 3px solid var(--color-accent);
  outline-offset: 3px;
}

.button:disabled {
  cursor: not-allowed;
  opacity: .45;
}

```

Como depurar sem adivinhar

1) confirme a URL e productId; 2) inspecione product no React DevTools; 3) confirme que o elemento existe no DOM; 4) pinte as caixas; 5) verifique regras CSS riscadas; 6) teste 1440px, 900px, 700px e 390px; 7) navegue apenas com Tab; 8) só então refine microespaçamentos.

12. Sequência prática para chegar ao resultado

Passo 1 — preserve o fluxo que funciona

Mantenha a rota `/products/:productId`, o hook de seleção e o callback do carrinho. Remova apenas o uso indevido de ProductCard styles e o className que recebe ProductStyle.

Passo 2 — crie vocabulário próprio

No estilo de ProductDetail, defina classes page, gallery, thumbnails, thumbnail, thumbnailActive, hero, info, metaRow, title, description, priceBlock, actions, quantity e buyButton. Nomes devem expressar responsabilidade, não aparência accidental.

Passo 3 — monte a árvore sem acabamento

Renderize todas as regiões com textos e botões. Use backgrounds temporários. Confirme que o produto correto aparece e que cada botão executa a ação esperada.

Passo 4 — resolva a grade externa

Aplique duas colunas no desktop e uma no breakpoint. Não ajuste tipografia antes de as regiões ocuparem a geometria correta.

Passo 5 — resolva a galeria

Crie gallery e activelImage, transforme imagens em botões e escolha cover ou contain conscientemente. Teste pelo menos uma imagem vertical e uma horizontal.

Passo 6 — componha a hierarquia comercial

Formate meta, título, estoque, avaliação, descrição, oferta, preço e parcelas. Use dados reais ou fallbacks explicitamente provisórios.

Passo 7 — conecte quantidade ao App

Mude o contrato do callback para transportar quantity. Centralize a validação de estoque no App e mantenha CartLine como a fonte global.

Passo 8 — valide estados e acessibilidade

Teste loading, 404, erro, sem estoque, teclado e foco. Adicione labels e desabilite ações impossíveis.

Passo 9 — valide o projeto

Execute lint e build. Depois compare desktop e mobile lado a lado com a referência, corrigindo primeiro proporção, depois ritmo e por último detalhes.

13. Esqueleto final do componente

O trecho abaixo reúne as decisões centrais sem fingir que os campos ainda inexistentes já chegam da API. Ele usa `imageUrl` como fallback, preserva os campos atuais e deixa pontos claros para evolução. Adapte o callback do carrinho em conjunto com o `App`; não mude apenas a assinatura local.

```
import { useState } from "react";
import { Product } from "../../types/product";
import useGetProduct from "../../hooks/useUrlParser";
import styles, { ProductStyle } from "../../style/components/Product";

interface ItemProps {
  products: Product[];
  onAddToCart: (id: string, quantity: number) => void;
}

const money = new Intl.NumberFormat("pt-BR", {
  style: "currency",
  currency: "BRL",
});

export default function ProductDetail({ products, onAddToCart }: ItemProps) {
  const product = useGetProduct(products);
  const gallery = product ? [product.imageUrl] : [];
  const [activeImage, setActiveImage] = useState(gallery[0] ?? "");
  const [quantity, setQuantity] = useState(1);

  if (!product) return <p>Produto não encontrado</p>;

  const stockLabel = product.stock === null
    ? "Sob encomenda"
    : `${product.stock} uni.`;

  return (
    <ProductStyle>
      <main className={styles.page}>
        <section className={styles.gallery} aria-label="Galeria do produto">
          <nav className={styles.thumbnails} aria-label="Imagens do produto">
            {gallery.map((image, index) => (
              <button
                key={image}
                type="button"
                className={styles.thumbnail}
                onClick={() => setActiveImage(image)}
                aria-label={`Mostrar imagem ${index + 1}`}
                aria-pressed={image === activeImage}
              >
                <img src={image} alt="" />
              </button>
            ))}
          </nav>
          <figure className={styles.hero}>
            <img src={activeImage || product.imageUrl} alt={product.alt || product.name} />
          </figure>
        </section>

        <article className={styles.info}>
          <div className={styles.metaRow}>
            <span>{product.categoryLabel} · Escala {product.scale}</span>
            <span className={styles.stock}>{stockLabel}</span>
          </div>
          <h1 className={styles.title}>{product.name}</h1>
          <p className={styles.material}>{product.material}</p>
          <strong className={styles.price}>{money.format(product.price)}</strong>

          <div className={styles.actionRow}>
            <div className={styles.quantity}>
              <button type="button" onClick={() => setQuantity(q => Math.max(1, q - 1))} aria-label="Diminuir quantidade">-</button>
              <output aria-live="polite">{quantity}</output>
              <button type="button" onClick={() => setQuantity(q => q + 1)} aria-label="Aumentar quantidade">+</button>
            </div>
            <button type="button" onClick={() => onAddToCart(product.id, quantity)} disabled={product.stock === 0}>
              {product.stock === 0 ? "Esgotado" : "Adicionar ao carrinho"}
            </button>
          </div>
        </article>
      </main>
    </ProductStyle>
  );
}
```

```

    </main>
  </ProductStyle>
);
}

```

Atenção sobre useState

Se gallery depender de dados que chegam depois, o estado inicial não é recalculado automaticamente. Em uma implementação completa, sincronize activeImage quando o product.id mudar ou remonte o componente com key. Esse detalhe é mais um motivo para distinguir loading de not found.

14. Checklist de conclusão

Camada	Pergunta de verificação
Rota	productId tem o mesmo nome na rota e em useParams?
Dados	A API fornece todos os textos e imagens ou os fallbacks estão explícitos?
Estrutura	Galeria e painel são irmãos; miniaturas e hero são filhos da galeria?
Grid	As colunas usam espaços, minmax e min-width: 0 sem overflow?
Imagem	O recorte foi escolhido conscientemente entre cover e contain?
Ações	A quantidade visível é a mesma enviada ao estado global?
Estoque	Zero desabilita; null representa sob encomenda; limite numérico é respeitado?
Responsivo	A hierarquia funciona em 1440, 900, 700 e 390px?
Acessibilidade	Há alt, aria-label, aria-pressed e foco visível?
Qualidade	npm run lint e npm run build terminam sem erro?

Definição de pronto

O resultado não está pronto apenas porque se parece com a referência em uma largura. Ele está pronto quando o produto correto vem da URL, os dados têm origem clara, o carrinho recebe a quantidade certa, a página continua utilizável sem mouse e o layout se reorganiza sem perder hierarquia.

Arquivos estudados

```

`src/components/ProductDetail/index.tsx`; `src/style/components/Product/index.ts`; `src/hooks/useUrlParser.tsx`; `src/routes/index.tsx`;
`src/App.tsx`; `src/services/api.ts`; `src/types/product.ts`; `src/style/theme.ts`; `src/style/GlobalStyles.ts`;
`src/components/ProductCard/index.tsx`.

```

Observação de escopo

Este PDF é um guia de implementação e raciocínio. Ele não altera os arquivos em edição do ProductDetail. Os mockups são ilustrações explicativas construídas com o asset real do projeto; não são capturas de um build validado no navegador.